

SYSTEM DESIGN — HLD NOTES

Video 1 + Video 2

PART 1 — INTRODUCTION TO SYSTEM DESIGN

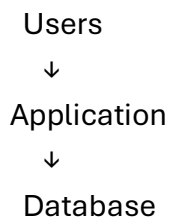
1. What is System Design?

System Design is the process of designing the architecture of a software system and deciding how its different components will communicate and work together.

The goal is to build a system that satisfies both:

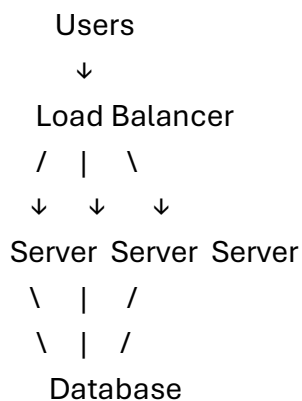
- **Functional requirements**
- **Non-functional requirements**

Example:



For a small application, this may be enough.

But when the application grows:



We need system design to decide **how to handle this growth**.

2. Functional Requirements

Functional requirements describe:

WHAT the system should do.

Example — Instagram:

- User can register/login
- User can upload a post
- User can follow another user
- User can like a post
- User can comment
- User can send messages

These are functionalities of the system.

Example

For a food-delivery system:

User can:

- ✓ Search restaurants
- ✓ View menu
- ✓ Place order
- ✓ Make payment
- ✓ Track order

3. Non-Functional Requirements

Non-functional requirements describe:

HOW WELL the system should work.

Important NFRs:

Scalability

Can the system handle increasing users/traffic?

Availability

Is the system accessible when users need it?

Reliability

Does the system consistently perform correctly?

Performance

How quickly does the system respond?

Security

Can we protect users and data from unauthorized access?

Maintainability

How easily can developers modify and maintain the system?

Functional vs Non-Functional

Functional	Non-Functional
What the system does	How well it does it
Login	Login should be fast
Upload video	Support millions of uploads
Place order	Order should be reliable
Send message	Message should have low latency
Make payment	Payment should be secure

4. HLD vs LLD

HLD — High-Level Design

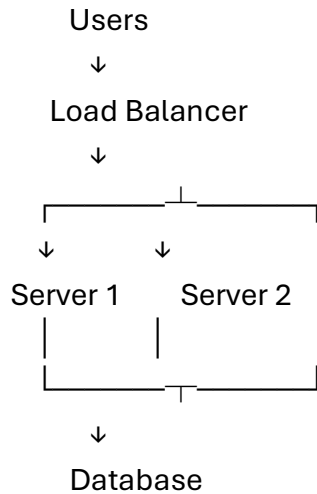
HLD describes the **overall architecture** of the system.

It focuses on:

- Components
- Services
- Databases
- APIs
- Load balancers
- Caches

- Message queues
- Communication
- Scalability
- Availability

Example:



LLD — Low-Level Design

LLD focuses on the **internal implementation of components**.

It includes:

- Classes
- Objects
- Interfaces
- Methods
- Design patterns
- Relationships between classes

Example:

User

```

├— id
├— name
├— email
├— login()
└— logout()
  
```

Easy way to remember

HLD = How the whole system is structured

LLD = How individual components are implemented

5. Scalability

Scalability is the ability of a system to handle increasing load by adding resources or changing its architecture.

There are two major types.

5.1 Vertical Scaling

Also called **Scaling Up**.

Increase the resources of an existing machine.

Before:

Server
4 CPU
8 GB RAM

↓

After:

Server
16 CPU
64 GB RAM

Advantages

- Simple
- Easy to implement
- Less architectural complexity

Disadvantages

- Hardware has a limit
- Can become expensive
- A single machine can become a bottleneck
- Doesn't solve the single-server failure problem

5.2 Horizontal Scaling

Also called **Scaling Out**.

Add more machines/instances.

Before:

Server 1

After:

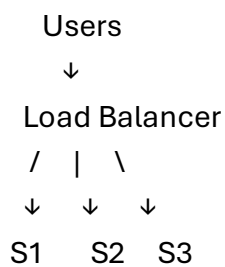
Server 1

Server 2

Server 3

Server 4

Usually combined with a **Load Balancer**:



Advantages

- Can handle large traffic
- Better fault tolerance
- Easier to increase capacity gradually

Disadvantages

- More complex
- Requires distributed-system techniques
- Data synchronization can become difficult

6. Latency vs Throughput

Latency

Time required to process a request.

Request → Server → Response

↑
Latency

Example:

API responds in 100 ms.

Throughput

Amount of work handled per unit of time.

Example:

Server handles 10,000 requests/second.

Remember

Latency → How fast?

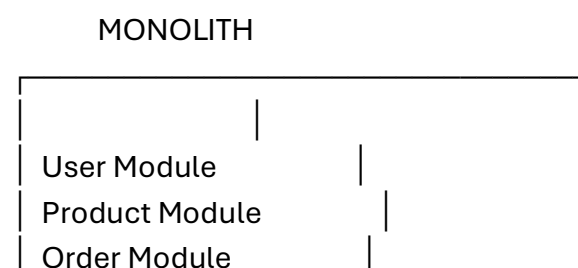
Throughput → How much?

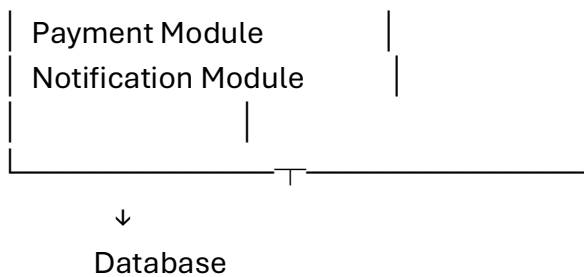
PART 2 — MONOLITHIC ARCHITECTURE

7. What is a Monolithic Application?

A **monolithic application** is an application where the major functionalities are packaged and deployed as **one application/unit**.

Example:





All modules may be logically separated in the code, but they belong to the same deployable application.

8. Characteristics of a Monolith

1. Single deployable unit

Usually the entire application is deployed together.

Code

↓

Build

↓

Test

↓

Deploy

↓

Entire Application

2. Modules are often tightly coupled

One module can directly depend on another module's:

- Code
- Logic
- Database tables
- Internal implementation

3. Centralized architecture

The application generally behaves as one large application.

4. Scaling is usually application-level

If one part requires more resources, you may have to scale the entire application.

9. Advantages of Monolithic Architecture

☒ Simple to develop initially

Everything is in one application.

☒ Easy local development

Developers don't need to run many services.

☒ Simple deployment

One application needs to be deployed.

☒ Easy communication

Modules can communicate through normal function/method calls instead of network calls.

Example:

```
orderService.createOrder()
```

rather than:

Order Service

↓ HTTP

Payment Service

☒ Easier debugging initially

The entire codebase is usually in one place.

10. Problems with Monolithic Architecture

10.1 Deployment Bottleneck

Suppose only the payment functionality changes.

You may still need:

Payment changed



Build entire application



Test entire application



Deploy entire application

10.2 Difficult Independent Scaling

Suppose:

User Module → Low traffic

Order Module → Medium traffic

Product Module → Very high traffic

You may need to scale the entire application instead of only Product.

Monolith



Scale everything

This can waste resources.

10.3 Large Codebase

As the application grows:

Small



Medium



Large



Huge monolith

It can become harder to:

- Understand
- Modify
- Test
- Deploy
- Maintain

10.4 Fault Isolation

A problem in one part of a monolithic application can potentially affect the entire application.

Therefore, isolating failures can be harder.

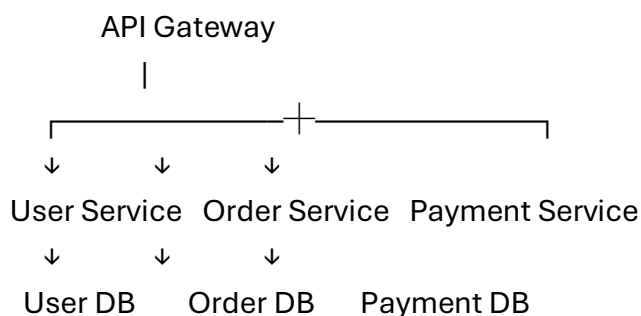
PART 3 — MICROSERVICES ARCHITECTURE

11. What are Microservices?

Microservices architecture divides an application into multiple small, independently deployable services.

Each service usually focuses on a particular **business capability**.

Example:



12. Characteristics of Microservices

12.1 Independent Deployment

Each service can be built and deployed independently.

User Service → Deploy independently

Order Service → Deploy independently

Payment Service → Deploy independently

If Payment Service changes, we don't necessarily need to redeploy User Service.

12.2 Independent Scaling

Suppose Payment Service receives huge traffic.

Payment Service



1 instance → 5 instances

Other services don't necessarily need to scale.

12.3 Fault Isolation

If one service fails:

Payment Service ❌

User Service ✅

Order Service ✅

Notification ✅

The entire system doesn't necessarily have to fail.

12.4 Independent Development

Different teams can own different services.

Team A → User Service

Team B → Order Service

Team C → Payment Service

This can improve team autonomy.

13. Microservices Communication

Microservices need to communicate with each other.

There are two major types.

13.1 Synchronous Communication

The caller waits for the response.

Order Service

|

| HTTP / REST / gRPC

↓

Payment Service

|

↓

Response

Example:

Order Service → "Process payment"

↓

Payment Service

↓

Response

↓

Order Service continues

Common technologies

- REST APIs
- gRPC

Problem

If Payment Service is slow:

Order Service

↓

waiting...

↓

Payment Service

The caller can also become slow.

If Payment Service is unavailable, the request may fail.

14. Asynchronous Communication

The sender sends a message/event and doesn't need to wait for immediate processing.

A message broker is commonly used.

Order Service



Message Broker



Notification Service

Examples:

- Kafka
- RabbitMQ

Example:

Order Created



Publish Event



Kafka



Notification Service



Send Email

The Order Service doesn't have to wait for the email operation.

15. Synchronous vs Asynchronous

Synchronous

Waits for response

Usually request-response

REST/gRPC

Simpler

Can have higher coupling

Caller can be blocked

Asynchronous

Doesn't need immediate response

Message/event based

Kafka/RabbitMQ

More complex

More loosely coupled

Caller can continue

Simple rule

Use **synchronous communication** when you need an immediate response.

Use **asynchronous communication** when the work can happen later/in the background.

PART 4 — MONOLITH → MICROSERVICES

16. Should We Immediately Convert a Monolith to Microservices?

No.

Converting an entire monolith at once is risky.

Bad approach:

Huge Monolith

↓

Rewrite everything

↓

Microservices

This is sometimes called a **Big Bang Rewrite**.

Instead, migrate gradually.

Existing Monolith

↓

Identify module

↓

Extract module

↓

Deploy service

↓

Monitor

↓

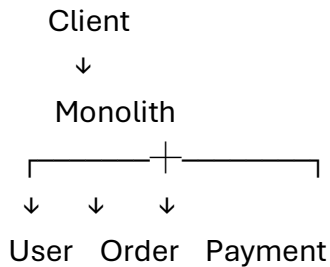
Repeat

17. Strangler Fig Pattern

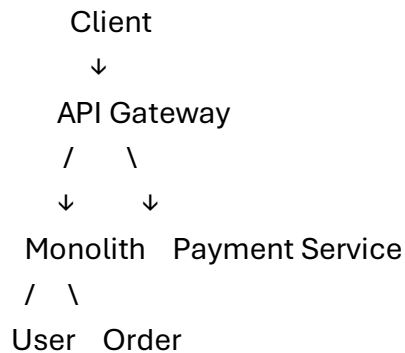
A common strategy for gradually migrating a monolith is the **Strangler Fig Pattern**.

The new architecture gradually takes over functionality from the old monolith.

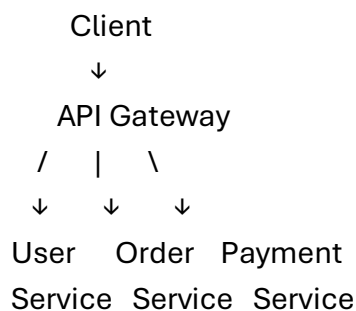
Initial system



Extract Payment:



Eventually:



The old monolith is gradually "strangled" by the new services.

18. Steps to Migrate Monolith → Microservices

Step 1 — Understand the monolith

Identify:

- Modules
- Dependencies
- Database dependencies
- Traffic patterns
- Critical components

Step 2 — Identify a good candidate

Choose a module that:

- Has clear boundaries
- Changes frequently
- Needs independent scaling
- Has high traffic
- Can be separated relatively safely

Step 3 — Define API Contract

Decide how the new service will communicate with other components.

Monolith

↓

API Contract

↓

New Service

Step 4 — Extract the module

Move its logic into a separate service.

Step 5 — Establish communication

Use:

REST / gRPC

or:

Kafka / RabbitMQ

depending on the use case.

Step 6 — Deploy and monitor

Monitor:

- Latency
- Error rate
- CPU
- Memory
- Traffic
- Availability

Step 7 — Gradually migrate more modules

Repeat the process.

PART 5 — CANARY DEPLOYMENT

19. What is Canary Deployment?

Canary deployment is a deployment strategy where a new version is gradually exposed to a small percentage of users/traffic before rolling it out to everyone.

Suppose:

Current Version → v1

New Version → v2

Don't immediately send:

100% → v2

Instead:

99.9% → v1

0.1% → v2

If v2 works correctly:

99% → v1

1% → v2

Then:

90% → v1

10% → v2

Eventually:

100% → v2

20. Why Canary Deployment?

Suppose v2 contains a serious bug.

With:

100% → v2

everyone can be affected.

With:

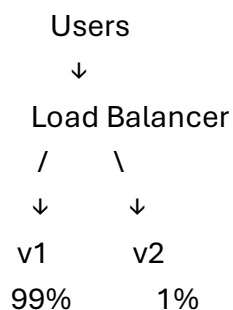
99.9% → v1

0.1% → v2

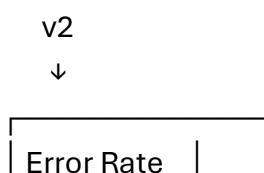
the impact is much smaller.

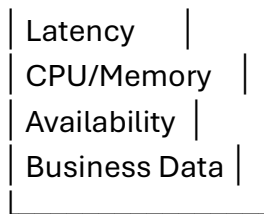
We can monitor v2 before increasing traffic.

21. Canary Deployment Architecture



Monitor v2:





If everything is good:

1% → 10% → 25% → 50% → 100%

If something goes wrong:

Rollback

↓

0% → v2

100% → v1

22. Other Deployment Strategies ★

These are important related concepts.

Rolling Deployment

Gradually replace old instances with new ones.

v1 v1 v1 v1

↓

v2 v1 v1 v1

↓

v2 v2 v1 v1

↓

v2 v2 v2 v1

↓

v2 v2 v2 v2

Blue-Green Deployment

Maintain two environments:

Blue → Current version

Green → New version

Initially:

Users → Blue

After testing:

Users → Green

If Green fails:

Users → Blue

Difference from Canary

Canary: gradually move a percentage of traffic.

Blue-Green: switch traffic between two environments.

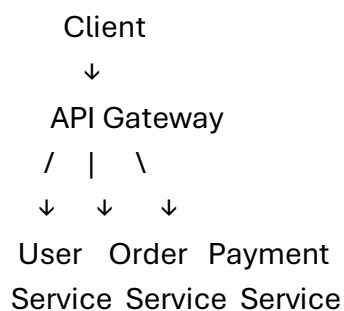
PART 6 — IMPORTANT RELATED CONCEPTS

These aren't things you need to master from Video #1–#2, but you should understand the basics because they directly connect to what you just learned.

23. API Gateway

In a microservices architecture, clients shouldn't necessarily communicate directly with every service.

Instead:



API Gateway can provide:

- Request routing
- Authentication/authorization
- Rate limiting

- Logging
- Request aggregation
- SSL/TLS termination

You'll study this properly in **Video #3**.

So for now:

API Gateway = entry point that routes client requests to appropriate services.

24. Service Discovery

In microservices, services may have multiple instances.

Payment Service

└─ Instance 1

└─ Instance 2

└─ Instance 3

Other services need to know:

"Where is the Payment Service?"

Service discovery helps services find available instances.

This becomes important when you start running many microservices.

25. Database-per-Service

A common microservices principle is:

User Service

↓

User DB

Order Service

↓

Order DB

Payment Service

↓

Payment DB

This provides better service independence.

But don't interpret this as:

"Every microservice **MUST** have a completely separate physical database."

The important idea is **data ownership and loose coupling**.

26. Distributed Transaction Problem

This is an important consequence of microservices.

In a monolith:

Order
Inventory
Payment
↓
Same Database

A database transaction can potentially cover multiple operations.

But in microservices:

Order Service
↓
Order DB

Inventory Service
↓
Inventory DB

Payment Service
↓
Payment DB

Now one business operation may involve **multiple databases**.

For example:

Create Order
↓
Reserve Inventory
↓
Process Payment

What happens if:

Create Order 

Reserve Inventory 

Payment 

We need a way to handle this distributed transaction.

This leads to the **Saga Pattern**.

27. Saga Pattern

Saga breaks a distributed transaction into multiple **local transactions**.

Example:

Create Order

↓

Reserve Inventory

↓

Process Payment

↓

Confirm Order

If Payment fails:

Payment 

↓

Release Inventory

↓

Cancel Order

The actions used to undo previously completed operations are called **compensating transactions**.

Two approaches

Choreography

Order Created

↓

Inventory

↓

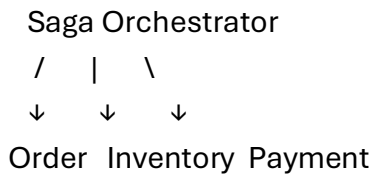
Inventory Reserved

↓

Payment

Services react to events.

Orchestration



A central orchestrator controls the workflow.

⚠ Don't go deep into Saga yet. Your playlist will build the distributed-system concepts needed for it.

28. Data Consistency

Microservices and distributed systems introduce another problem:

How do we keep data consistent across different services/databases?

Two important concepts:

Strong Consistency

After a write, reads see the latest value.

Useful when correctness is critical.

Example:

Bank Balance

Payment Status

Eventual Consistency

Different replicas/services may temporarily have different data, but they eventually become consistent.

Example:

User likes post

↓

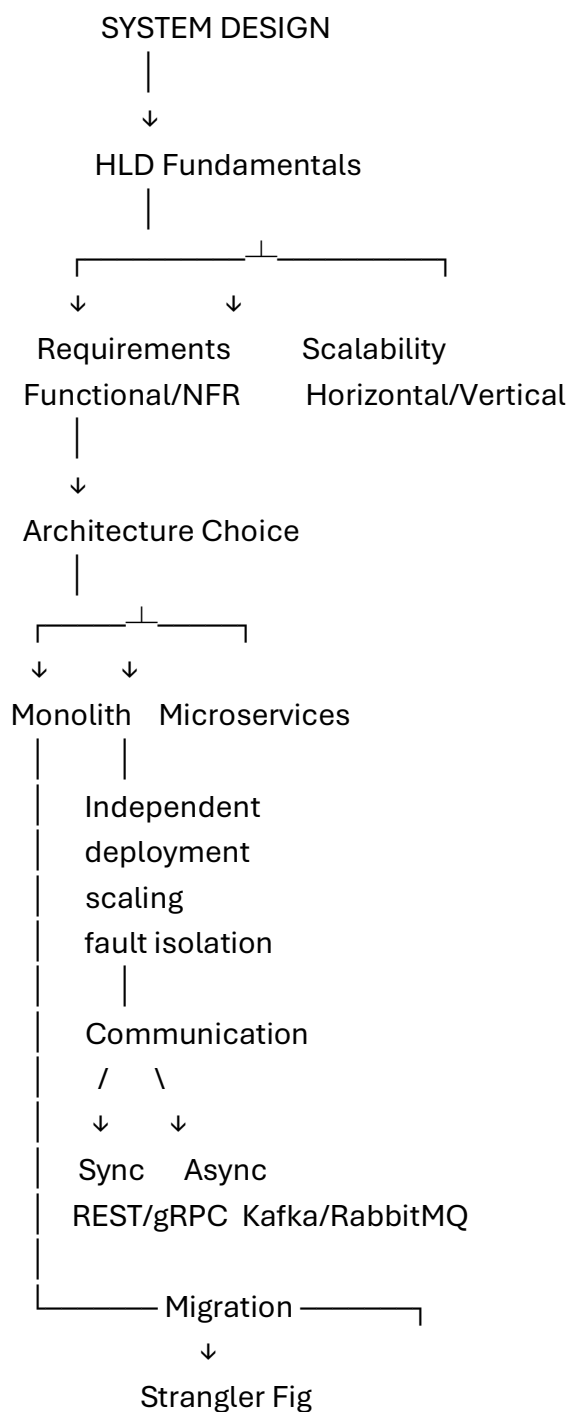
Database

↓
Other replicas
↓
Eventually updated

You'll study consistency much more deeply with **distributed systems + CAP theorem** later.

THE BIG PICTURE

After Video #1 + #2, you should understand this progression:



★ What You MUST Know After Video 1 + 2

If you're revising before continuing, make sure you can explain these **without looking at your notes**:

HLD Fundamentals

- What is System Design?
- HLD vs LLD
- Functional vs Non-functional requirements
- Scalability
- Horizontal vs Vertical scaling
- Latency vs Throughput
- Availability
- Reliability
- Performance

Architecture

- What is a Monolith?
- Advantages of Monolith
- Problems with Monolith
- What are Microservices?
- Advantages of Microservices
- Independent deployment
- Independent scaling
- Fault isolation
- Why microservices increase complexity

Communication

- Synchronous communication
- Asynchronous communication
- REST
- gRPC
- Message queues
- Kafka/RabbitMQ — basic purpose

Migration & Deployment

- Why not rewrite a monolith in one shot?
- Strangler Fig Pattern
- Canary Deployment
- Rolling Deployment
- Blue-Green Deployment

Microservices Concepts

- API Gateway — basic idea
- Service Discovery — basic idea
- Database-per-service — basic idea
- Distributed transaction problem
- Saga Pattern — basic idea
- Strong vs Eventual Consistency — basic idea